

Employee Management System

Project Documentation

React + Spring Boot + MySQL Full-Stack CRUD Application

Architecture, Build Process, and Setup Guide

Version 1.0

Table of Contents

Table of Contents.....	2
1. Project Overview	3
Key Features	3
2. Technology Stack	3
3. System Architecture	3
3.1 Frontend Architecture.....	4
3.2 Backend Architecture	4
3.3 Data Flow Example: Creating an Employee.....	4
4. Project Structure	5
5. Data Model	5
6. REST API Reference	6
Error Response Format	6
7. Frontend Routes.....	6
8. How the Project Was Built	7
8.1 Backend Build Steps.....	7
8.2 Frontend Build Steps	7
8.3 Containerization.....	7
9. Step-by-Step Setup Guide.....	8
9.1 Option A — Run with Docker (Recommended)	8
9.2 Option B — Run Manually (Local Installs).....	8
10. Configuration Notes	8
11. Known Limitations and Future Enhancements	9

1. Project Overview

The Employee Management System is a full-stack CRUD (Create, Read, Update, Delete) web application for managing employee records. It provides a browser-based interface for HR/admin users to add, view, search, update, and delete employees, backed by a REST API and a relational database.

Each employee record captures: first name, last name, email address, phone number, department, designation, date of joining, and salary. Both the client and server validate this data, and the server rejects duplicate email addresses.

Key Features

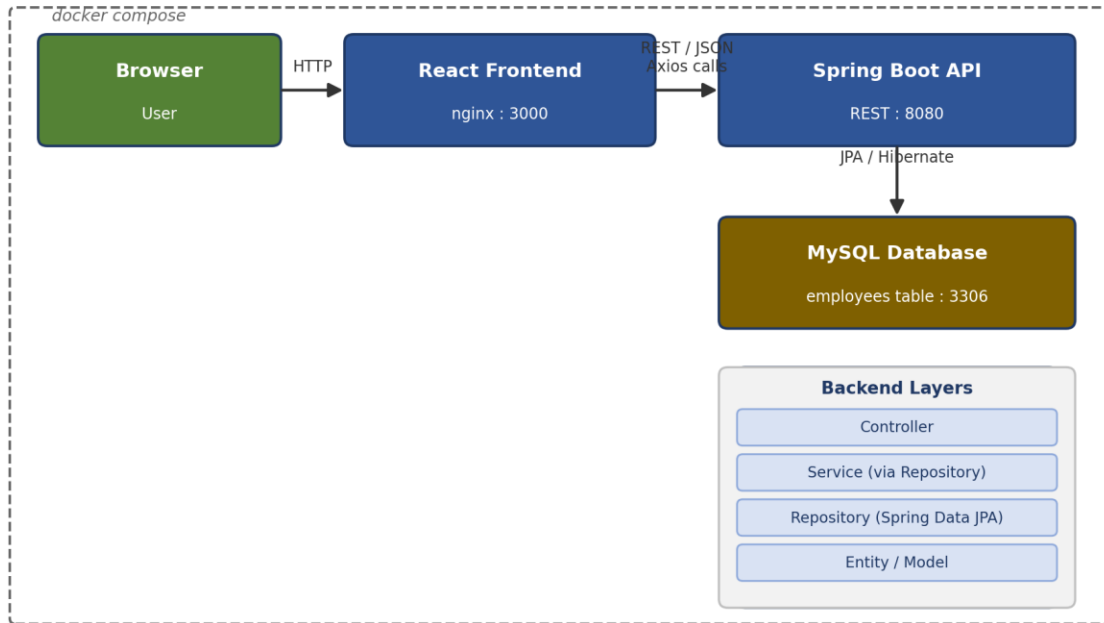
- List all employees in a searchable, sortable table
- Add a new employee via a validated form
- View full details of a single employee
- Update an existing employee's details
- Delete an employee with a confirmation prompt
- Server-side validation with field-level error messages
- Duplicate-email protection (HTTP 409 Conflict)
- One-command startup for the entire stack via Docker Compose

2. Technology Stack

Layer	Technology	Purpose
Frontend	React 16 + React Router 5	Single-page application UI and client-side routing
Frontend	Axios	HTTP client for calling the REST API
Frontend	Bootstrap 4	Layout and styling
Backend	Spring Boot 3.0.4 (Java 17)	REST API server
Backend	Spring Data JPA / Hibernate	Object-relational mapping to MySQL
Backend	Jakarta Bean Validation	Server-side field validation
Database	MySQL 8	Persistent storage of employee records
Build Tools	Maven (backend), npm (frontend)	Dependency management and builds
Deployment	Docker + Docker Compose	Containerized, one-command startup

3. System Architecture

The system follows a classic three-tier architecture: a React single-page application (presentation tier), a Spring Boot REST API (application/business logic tier), and a MySQL database (data tier). In the Docker Compose setup, each tier runs in its own container on a shared network.



3.1 Frontend Architecture

The React app is organized around a small set of class components, each responsible for one screen:

- **ListEmployeeComponent** — fetches and displays all employees; provides client-side search/filter and triggers add/edit/view/delete actions
- **CreateEmployeeComponent** — a single form used for both adding a new employee and editing an existing one (mode is determined by the `:id` route parameter); performs client-side validation before calling the API
- **ViewEmployeeComponent** — read-only detail view for one employee
- **HeaderComponent / FooterComponent** — static page chrome
- **EmployeeService** — a single module wrapping all Axios calls to the backend, so components never call axios directly

Routing is handled by React Router 5 in `App.js`, mapping URL paths to the components above.

3.2 Backend Architecture

The Spring Boot backend follows a standard layered structure:

- **Controller** (`EmployeeController`) — exposes REST endpoints, delegates to the repository, and performs request-level checks such as duplicate-email detection
- **Repository** (`EmployeeRepository`) — a Spring Data JPA interface providing CRUD operations and custom existence checks (`existsByEmailId`, `existsByEmailIdAndIdNot`) with no manual SQL required
- **Entity/Model** (`Employee`) — the JPA-mapped class representing the employees table, annotated with Jakarta Bean Validation constraints
- **Exception handling** (`GlobalExceptionHandler`, `ResourceNotFoundException`, `EmailAlreadyExistsException`) — centralizes error handling so every error returns a consistent JSON structure with an appropriate HTTP status code

3.3 Data Flow Example: Creating an Employee

1. The user fills out the Add Employee form in the browser and clicks Save.

2. CreateEmployeeComponent validates the fields client-side; if invalid, it shows inline errors and stops.
3. If valid, EmployeeService sends a POST request to /api/v1/employees with the employee JSON payload.
4. EmployeeController receives the request, checks for a duplicate email via the repository, and if none exists, saves the entity.
5. Spring Data JPA / Hibernate translates the save into an SQL INSERT against the MySQL employees table.
6. The saved employee (now including its generated ID) is returned as JSON with HTTP 201 Created.
7. The frontend redirects back to the employee list, which re-fetches all employees.

4. Project Structure

```
ReactJS-Spring-Boot-CRUD-Full-Stack-App-master/
+-- docker-compose.yml
+-- DOCUMENTATION.md
+-- react-frontend/
|   +-- Dockerfile
|   +-- package.json
|   +-- src/
|       +-- App.js
|       +-- components/
|           +-- ListEmployeeComponent.jsx
|           +-- CreateEmployeeComponent.jsx
|           +-- ViewEmployeeComponent.jsx
|           +-- HeaderComponent.js
|           +-- FooterComponent.jsx
|       +-- services/
|           +-- EmployeeService.js
+-- springboot-backend/
    +-- Dockerfile
    +-- pom.xml
    +-- src/main/java/net/javaguides/springboot/
        +-- SpringbootBackendApplication.java
        +-- controller/EmployeeController.java
        +-- model/Employee.java
        +-- repository/EmployeeRepository.java
        +-- exception/
            +-- ResourceNotFoundException.java
            +-- EmailAlreadyExistsException.java
            +-- GlobalExceptionHandler.java
```

5. Data Model

The core entity is Employee, mapped to the employees table:

Field	Type	Validation Rule
id	Long (PK)	Auto-generated identity
firstName	String	Required, max 50 characters
lastName	String	Required, max 50 characters

Field	Type	Validation Rule
emailId	String	Required, valid email format, unique across all employees
department	String	Required, max 50 characters
designation	String	Required, max 50 characters
phoneNumber	String	Required, must match a valid phone number pattern
dateOfJoining	LocalDate	Required, cannot be a future date
salary	BigDecimal	Required, must be greater than 0

6. REST API Reference

Base URL: <http://localhost:8080/api/v1>

Method	Endpoint	Description	Success Status
GET	/employees	List all employees	200 OK
POST	/employees	Create a new employee	201 Created
GET	/employees/{id}	Get one employee by ID	200 OK
PUT	/employees/{id}	Update an employee by ID	200 OK
DELETE	/employees/{id}	Delete an employee by ID	200 OK

Error Response Format

All errors are handled centrally by `GlobalExceptionHandler` and return a consistent JSON structure:

```
{
  "timestamp": "2026-09-06T10:15:30",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed. Please check the highlighted fields.",
  "fieldErrors": {
    "emailId": "Email must be a valid email address",
    "salary": "Salary must be greater than 0"
  }
}
```

Status	Trigger
400 Bad Request	Bean validation failure (missing or invalid fields)
404 Not Found	Employee ID does not exist
409 Conflict	Email address already in use (on create or update)
500 Internal Server Error	Unexpected server-side error

7. Frontend Routes

Path	Component	Purpose
/ and /employees	ListEmployeeComponent	Employee list with search
/add-employee/_add	CreateEmployeeComponent	Add a new employee
/add-employee/:id	CreateEmployeeComponent	Edit an existing employee
/view-employee/:id	ViewEmployeeComponent	Read-only employee details

8. How the Project Was Built

This section walks through the order in which the application was constructed, which is also the recommended order to follow if extending it.

8.1 Backend Build Steps

1. Bootstrap a Spring Boot project (Spring Initializr) with the Web, JPA, MySQL Driver, and Validation dependencies, targeting Java 17.
2. Define the Employee entity in model/Employee.java, mapping it to the employees table and annotating each field with Jakarta Bean Validation constraints (@NotBlank, @Email, @Pattern, @PastOrPresent, @DecimalMin, etc.).
3. Create EmployeeRepository extending JpaRepository<Employee, Long>, adding existsByEmailId and existsByEmailIdAndIdNot for duplicate-email checks.
4. Implement EmployeeController with endpoints for list, create, get-by-id, update, and delete, using @Valid on the request body for create/update.
5. Add custom exceptions (ResourceNotFoundException, EmailAlreadyExistsException) and a GlobalExceptionHandler (@RestControllerAdvice) to convert them, along with validation errors, into a consistent JSON error shape.
6. Configure application.properties with the MySQL connection URL, credentials, and spring.jpa.hibernate.ddl-auto=update so tables are created/updated automatically.
7. Enable CORS for the frontend's origin (http://localhost:3000) via @CrossOrigin on the controller.

8.2 Frontend Build Steps

1. Scaffold a React app (Create React App) and add axios, react-router-dom, and bootstrap as dependencies.
2. Create EmployeeService.js to centralize all HTTP calls to the backend's /api/v1/employees endpoints.
3. Build ListEmployeeComponent to fetch and render all employees in a table, with search input and action buttons (View, Update, Delete).
4. Build CreateEmployeeComponent as a shared add/edit form: it reads the :id route parameter, loads existing data in edit mode, validates all fields client-side, and calls the appropriate service method.
5. Build ViewEmployeeComponent to display a single employee's full details in read-only form.
6. Wire up routes in App.js with React Router, and add HeaderComponent / FooterComponent for consistent page chrome.
7. Style the app with Bootstrap classes and a small amount of custom CSS.

8.3 Containerization

1. Write a multi-stage Dockerfile for the backend: build the jar with Maven in one stage, then run it on a lightweight JRE image.
2. Write a multi-stage Dockerfile for the frontend: build the static React bundle with Node, then serve it with nginx.
3. Write docker-compose.yml defining three services — db (MySQL with a health check), backend (depends on a healthy db), and frontend (depends on backend) — each with their ports mapped to the host.

9. Step-by-Step Setup Guide

9.1 Option A — Run with Docker (Recommended)

This is the simplest path: the only software you install locally is Docker Desktop.

1. Install Docker Desktop from docker.com and make sure it is running (on Windows, this requires WSL2, which Docker Desktop can install for you).
2. Unzip the project folder and open a terminal inside the top-level folder that contains docker-compose.yml.
3. Run: `docker compose up --build`
4. Wait for the logs to show the backend has started and the frontend's nginx server is running.
5. Open `http://localhost:3000` in your browser to use the application.
6. To stop the application, press Ctrl+C, then run: `docker compose down` (add `-v` to also delete the database volume).

9.2 Option B — Run Manually (Local Installs)

Prerequisites: Java 17, Maven (or the included mvnw wrapper), Node.js with npm, and a running MySQL Server.

1. Create the database: `mysql -u root -p -e "CREATE DATABASE employee_management_system;"`
2. Open `springboot-backend/src/main/resources/application.properties` and set your MySQL username and password.
3. From the `springboot-backend` folder, run: `./mvnw spring-boot:run` (or `mvnw.cmd spring-boot:run` on Windows). The API starts on port 8080.
4. Confirm the API is up by visiting `http://localhost:8080/api/v1/employees` in a browser.
5. In a second terminal, go to the `react-frontend` folder and run: `npm install`
6. Then run: `npm start`. This opens `http://localhost:3000` automatically.
7. Use the app: click Add Employee to create a record, then View, Update, or Delete it from the list.

10. Configuration Notes

- CORS: the backend only accepts requests from `http://localhost:3000` by default (see `@CrossOrigin` in `EmployeeController`). Update this if the frontend is served from a different origin.
- Schema management: `spring.jpa.hibernate.ddl-auto=update` automatically creates/updates the employees table on startup — no manual migration scripts are needed for local development.
- Date serialization: `spring.jackson.serialization.write-dates-as-timestamps=false` ensures `dateOfJoining` is returned as an ISO date string (`yyyy-MM-dd`) rather than a numeric array.
- Docker networking: inside `docker-compose.yml`, the backend connects to the database using the service name `db` as the hostname (`jdbc:mysql://db:3306/...`), not `localhost`.

11. Known Limitations and Future Enhancements

- No authentication or authorization — anyone with network access to the API can view or modify data.
- No pagination on the employee list; all records are fetched at once.
- Search is client-side only (filters already-loaded data) rather than a backend query, which will not scale well to very large employee counts.
- UpdateEmployeeComponent.jsx exists in the codebase but is unused; editing is handled by CreateEmployeeComponent in an 'edit mode' driven by the :id route parameter.
- No automated frontend tests; the backend has only the default Spring Boot context-load test.
- Suggested next steps: add JWT-based authentication, backend pagination and search endpoints, role-based access control, and a CI pipeline that builds and tests both the frontend and backend on every push.